FarmOS Frontend API Integration Guide

Domain 01 governance and API v2.2 | Development correction addendum: 2026-10-13

Scope: this guide covers the locked Domain 01 correction, implemented Domain 02 contracts, and the development-backend correction addendum. Recent addendum coverage includes groups and membership history, farm labels, animal gallery media, health-observation lifecycle, schedule windows and generated-task history, animal/assignee selectors, My Work, and mobile field workflows. It is an integration guide and does not replace fuller domain-specific API references.

Status: current development-backend handoff only. Automated backend verification passed as recorded in the evidence report. Frontend implementation and acceptance are outstanding, the overall product is not locked, and no changes have been promoted to the live backend.

Backend authorization remains authoritative; the frontend must not infer permission from role names.

1. Integration sequence

Follow this order after the user opens the application:

1. Send the channel header. 2. Log in. 3. Store cookies automatically. 4. Fetch the current user profile. 5. Fetch capabilities and navigation. 6. Render only permitted governance screens. 7. Call organization/farm endpoints with authorized identifiers. 8. Handle structured error codes.

Required headers

```
{
  "Content-Type": "application/json",
  "X-App-Channel": "web",
  "X-CSRFToken": "value-from-client_csrf_token"
}
```

Use X-App-Channel: mobile for the mobile client. Do not store access or refresh tokens in browser localStorage; the backend uses secure cookies.

2. Standard response envelopes

Every v2 endpoint returns a predictable JSON envelope. Read data from data and pagination information from meta.

```
{
  "success": true,
  "code": "REQUEST_SUCCESSFUL",
  "message": "Request completed successfully.",
  "data": {
    "id": 123
  },
  "meta": null
}
{
  "success": false,
  "code": "PERMISSION_DENIED",
  "message": "You do not have permission to perform this action.",
  "data": null,
  "errors": {},
  "retryable": false
}
```

3. Authentication and bootstrap

POST **/api/auth/login**

Purpose: Authenticate an active user and create the secure Web/Mobile session.

Request JSON

```
{
  "email": "owner@example.com",
  "password": "your-password"
}
```

Response JSON

```
{
  "status": "Success",
  "message": "Login successful",
  "is_admin": false
}
```

Frontend notes: The response sets client_access_token, client_refresh_token, and client_csrf_token cookies. A deactivated account returns ACCOUNT_DEACTIVATED.

GET **/api/v2/users/me/**

Purpose: Load the signed-in user identity, organization, ownership, assignments, and channel access.

Response JSON

```
{
  "success": true,
  "code": "REQUEST_SUCCESSFUL",
  "data": {
    "identity": {
      "id": "uuid",
      "display_name": "Ada Farmer",
      "email": "owner@example.com"
    },
    "access": {
      "account_type": "organization_owner",
      "access_source": "organization_ownership",
      "scope": "organization",
      "is_organization_owner": true,
      "all_farms": true,
      "assignments": []
    },
    "channel_access": {
      "web": true,
      "mobile": false
    }
  }
}
```

Frontend notes: Use this response to build the account shell. Owner status is a boolean/context decision, not a role-name comparison.

GET **/api/v2/users/me/capabilities/**

Purpose: Load effective capabilities and navigation primitives.

Response JSON

```
{
  "success": true,
  "data": {
    "permissions": [
      "view_animal_details",
      "view_operation"
    ],
    "capabilities": {
      "view_animal_details": true,
      "create_operation": false
    },
    "navigation": {
      "livestock": true,
      "operations": true,
```

```
      "people": false
    },
    "channel_access": {
      "web": true,
      "mobile": true
    }
  }
}
```

Frontend notes: Render actions from capabilities. Do not assume Farm Manager or Veterinarian authority from labels.

POST **/api/auth/refresh-token**

Purpose: Renew an expired access session using the refresh cookie.

Response JSON

```
{
  "success": true,
  "message": "Token refreshed"
}
```

Frontend notes: If renewal fails, redirect to login. Deactivation cannot be bypassed through refresh.

POST **/api/auth/signout**

Purpose: Terminate the current authenticated session.

Response JSON

```
{
  "success": true,
  "message": "Logged out successfully"
}
```

Frontend notes: After logout, protected requests must fail; the account remains eligible for a fresh login.

4. Organization and people

GET **/api/v2/users/?page=1&page;_size=20**

Purpose: List members of the authenticated organization.

Response JSON

```
{
  "success": true,
  "data": [
    {
      "id": "uuid",
      "display_name": "Ada Farmer",
      "email": "owner@example.com",
      "account_status": "active",
      "owner": true
    }
  ],
  "meta": {
    "pagination": {
      "page": 1,
      "page_size": 20,
      "total_items": 1,
      "total_pages": 1
    }
  }
}
```

Frontend notes: The owner field is a boolean. Search with ?search=. Never filter another organization in the frontend.

PATCH **/api/v2/users/me/**

Purpose: Update permitted personal profile fields only.

Request JSON

```
{
  "display_name": "Ada Farmer",
  "phone": "08012345678"
}
```

Response JSON

```
{
  "success": true,
  "code": "PROFILE_UPDATED",
  "message": "Profile updated successfully.",
  "data": {
    "user": {
      "display_name": "Ada Farmer",
      "phone": "08012345678"
    }
  }
}
```

Frontend notes: Any authenticated active user may update their own display_name and phone. Do not send role, permissions, farm_assignment, account_status, or organization_ownership; those fields are rejected.

POST /api/v2/users/me/avatar/

Purpose: Upload or replace the current user's profile picture using multipart/form-data.

Request JSON

```
"multipart/form-data: avatar=@profile.jpg (JPG or PNG, maximum 5 MB)"
```

Response JSON

```
{
  "success": true,
  "code": "PROFILE_IMAGE_UPDATED",
  "message": "Profile picture updated successfully.",
  "data": {
    "avatar_url": "https://api.example.com/media/avatars/profile.jpg"
  }
}
```

Frontend notes: Any authenticated active user may manage their own picture. Send the file as avatar; do not send JSON for this request.

DELETE /api/v2/users/me/avatar/

Purpose: Remove the current user's profile picture without deleting the account or profile.

Response JSON

```
{
  "success": true,
  "code": "PROFILE_IMAGE_REMOVED",
  "message": "Profile picture removed successfully.",
  "data": {
    "avatar_url": null
  }
}
```

Frontend notes: The operation affects only the signed-in user's avatar. Deactivated users cannot call protected profile endpoints.

GET /api/v2/organizations/{organization_id}/

Purpose: Read the organization profile and ownership context.

Response JSON

```
{
  "success": true,
  "data": {
    "id": "uuid",
    "name": "Green Valley Farm",
    "country": "Nigeria",
    "state_region": "Lagos",
    "owner": {
      "id": "uuid",
      "is_owner": true
```

```
    }
  }
}
```

5. Account-state lifecycle

Account state is evaluated on login, protected requests, and refresh. The lifecycle is ACTIVE → DEACTIVATED → REACTIVATED. Deactivation preserves profile, assignment history, and audit history; it does not delete the user.

```
{
  "account_status": "active"
}
```

POST **/api/v2/users/{user_id}/deactivate/**

Purpose: Deactivate a staff account from an authorized governance context.

Request JSON

```
{
  "reason": "No longer employed"
}
```

Response JSON

```
{
  "success": true,
  "data": {
    "user_id": "uuid",
    "account_status": "deactivated"
  }
}
```

Frontend notes: Existing sessions fail on the next protected request. Fresh login and refresh return ACCOUNT_DEACTIVATED.

POST **/api/v2/users/{user_id}/reactivate/**

Purpose: Restore account eligibility without recreating revoked assignments.

Response JSON

```
{
  "success": true,
  "data": {
    "user_id": "uuid",
    "account_status": "active"
  }
}
```

Frontend notes: The user may log in again; effective access is recalculated from current membership and assignments.

6. Audit and security events

The backend emits durable security/access events. The frontend should display structured event fields and must never parse a generic summary string.

```
{
  "event_name": "USER_ASSIGNMENT_UPDATED",
  "actor_id": "uuid",
  "target_id": "assignment-id",
  "organization_id": "uuid",
  "farm_id": 1,
  "timestamp": "2026-09-16T08:00:00Z",
  "previous_state": {
    "farm_id": 1
  },
  "new_state": {
    "farm_id": 2
  },
  "result": "success",
  "source": "api"
}
```

Implemented event names include USER_LOGGED_IN, USER_LOGGED_OUT, USER_DEACTIVATED, USER_REACTIVATED, USER_ASSIGNMENT_CREATED, USER_ASSIGNMENT_UPDATED,

USER_ASSIGNMENT_REVOKED, ROLE_CREATED, ROLE_UPDATED, ROLE_PERMISSIONS_UPDATED, ORGANIZATION_CREATED, and FARM_CREATED.

Evidence: the development test suite verifies login, assignment, deactivation/reactivation, audit context, timestamps, result/source fields, and exclusion of passwords, access tokens, refresh tokens, and CSRF secrets.

7. Organization farms

GET	/api/v2/organizations/{organization_id}/farms/

Purpose: List farms visible to the current user.

Response JSON

```
{
  "success": true,
  "data": [
    {
      "id": 1,
      "name": "North Farm",
      "farm_code": "FRM-001",
      "status": "active"
    }
  ],
  "meta": {
    "pagination": {
      "page": 1,
      "page_size": 20,
      "total_items": 1
    }
  }
}
```

Frontend notes: Use returned farm IDs for subsequent scoped requests.

GET	/api/v2/farms/{farm_id}/

Purpose: Read a farm profile and scoped counts.

Response JSON

```
{
  "success": true,
  "data": {
    "id": 1,
    "organization_id": "uuid",
    "name": "North Farm",
    "status": "active",
    "counts": {
      "animals": 42,
      "open_tasks": 3
    }
  }
}
```

8. Roles, assignments, and channel rules

Role and assignment administration is available to owners or users with the relevant governance capability.

```
{
  "staff_role_templates": {
    "Farm Manager": {
      "web": true,
      "mobile": true
    },
    "Veterinarian": {
      "web": true,
      "mobile": true
    },
    "Field Worker": {
      "web": false,
      "mobile": true
    }
  }
}
```

Organization Owner is deliberately absent from role_templates: Owner authority comes from organization ownership, not a staff role.

Assignment payload:

```
{
  "user_id": "uuid",
```

```
  "role_id": 3,
  "farm_id": 1,
  "client_request_id": "assignment-001"
}
```

A revoked assignment remains in history but no longer grants current farm authority. A Field Worker calling a Web-protected endpoint receives CHANNEL_ACCESS_DENIED.

9. Direct domain actions (D02-COR-001)

Operations coordinates planned and assigned work; it is not a universal gateway for legitimate farm activity. Use task completion only when the user is executing an existing task. For an immediate authorized action, call the direct endpoint and do not create a placeholder task.

POST	/api/v2/domain-actions/

Purpose: Create the authoritative record for an authorized ad-hoc domain action without creating an Operations task.

Request JSON

```
{
  "farm_id": 1,
  "animal_id": 42,
  "task_type": "vaccination",
  "vaccine_name": "FMD",
  "date_given": "2026-09-23",
  "client_request_id": "mobile-fmd-001"
}
```

Response JSON

```
{
  "success": true,
  "code": "REQUEST_SUCCESSFUL",
  "data": {
    "execution_context": "direct",
    "task": null,
    "subject": {
      "type": "animal",
      "id": 42,
      "label": "ANIMAL-042"
    },
    "result": {
      "type": "vaccination_record",
      "id": 123
    }
  }
}
```

Frontend notes: Supported task_type values: vaccination, treatment, feed_issuance, sale, movement, observation, weight, pregnancy_check, and mortality. generic is not a direct action because it has no authoritative domain result.

Authorization is evaluated against the required domain capability and authorized farm scope, independently of Operations capabilities. The endpoint uses the same validation, timeline/audit events, inventory or animal-state effects, follow-up generation, and idempotency safeguards as typed task completion.

When completing an existing task, continue to use POST /api/v2/operations/tasks/{task_id}/complete/. Its resulting business record is linked from the task. Do not submit the same work through both paths.

10. Task planning and subject model (D02-002)

Create tasks in this order: Farm, task type, permitted subject, optional individual assignee, due datetime, priority, then instructions. The server validates farm scope, subject compatibility, active assignee eligibility, and priority. Do not offer role-targeted assignment.

POST	/api/v2/operations/tasks/

Purpose: Create planned work. Typed titles are derived by the backend; generic tasks require a meaningful title.

Request JSON

```
{
  "farm_id": 1,
  "task_type": "vaccination",
```

```
  "animal_id": 42,
  "assignee_id": "uuid",
  "due_at": "2026-09-24T09:00:00+01:00",
  "priority": "high",
  "description": "Use cattle crush near Pen 3."
}
```

Response JSON

```
{
  "success": true,
  "data": {
    "task_type": "vaccination",
    "title": "Vaccinate ANIMAL-042",
    "status": "assigned"
  }
}
```

Frontend notes: Canonical types: vaccination, treatment, feed_issuance, observation, weight, movement, pregnancy_check, sale, mortality, and generic. Treatment, observation, weight, pregnancy check, sale, and mortality require an individual animal; pregnancy check requires a female animal. Vaccination, feed issuance, and movement accept either an animal or group. generic may be farm-scoped without a subject.

GET `/api/v2/operations/tasks/{task_id}/vaccination-roster/?page=1&page;_size=50`

Purpose: Read the current group vaccination execution roster with backend eligibility and exclusion reasons.

Response JSON

```
{
  "success": true,
  "data": [
    {
      "id": 42,
      "tag_id": "COW-042",
      "name": "Borno Bull Alpha",
      "eligible": true,
      "eligibility_reason": null
    },
    {
      "id": 43,
      "tag_id": "COW-043",
      "name": "COW-043",
      "eligible": false,
      "eligibility_reason": {
        "code": "DOMAIN_STATE_CHANGED",
        "message": "Animal is no longer active in this Farm."
      }
    }
  ]
}
```

Frontend notes: Requires task execution and domain capabilities. The roster contains current active group memberships in the Task Farm. Completion must still submit executed_animal_ids and excluded_animal_ids; the backend revalidates live state at submission.

GET `/mobile/api/tasks/{task_id}/vaccination-roster/?page=1&page;_size=50`

Purpose: Read the assigned mobile worker's current group vaccination roster with backend eligibility and exclusion reasons.

Frontend notes: Uses the mobile JWT session and only returns data for the assigned worker's Task.

11. Scheduling and recurring work (D02-007)

A Schedule is a farm-scoped reusable work-generation rule, not a Task. Its state is active/inactive; every run creates a normal independently traceable Operations Task with source.type=schedule and source.id set to the schedule ID. Deactivating a schedule stops future generation only; it does not cancel or rewrite prior tasks.

POST `/api/v2/operations/schedules/`

Purpose: Create an active farm-scoped schedule. Set run_now true only when the user understands it generates a Task immediately and advances the schedule.

Request JSON

```
{
  "farm_id": 1,
  "task_type": "vaccination",
  "title": "Monthly CBPP round",
  "animal_id": 42,
  "recurrence": "monthly",
  "next_run_at": "2026-10-01T07:00:00+01:00",
  "assignee_id": "uuid",
  "run_now": false
}
```

Response JSON

```
{
  "success": true,
  "data": {
    "id": 17,
    "task_type": "vaccination",
    "title": "Monthly CBPP round",
    "recurrence": "monthly",
    "next_run_at": "2026-10-01T07:00:00+01:00",
    "is_active": true
  }
}
```

Frontend notes: Allowed recurrence values are once, daily, weekly, and monthly. The server validates the subject and assignee with the same farm/access rules as generated Tasks. Schedule title is explicit and is retained by generated tasks.

POST **/api/v2/operations/schedules/{schedule_id}/run/**

Purpose: Manually generate the current occurrence from an existing schedule.

Response JSON

```
{
  "success": true,
  "data": {
    "id": 304,
    "title": "Monthly CBPP round",
    "status": "assigned",
    "source": {
      "type": "schedule",
      "id": 17
    }
  }
}
```

Frontend notes: Manual Run remains schedule-sourced while the triggering user is retained as the task/event actor. It advances next_run_at; for once it deactivates the schedule. Confirm this consequence in the UI.

PATCH **/api/v2/operations/schedules/{schedule_id}/**

Purpose: Update supported future-template fields, including title, description, recurrence, next_run_at, subject, assignee, and is_active.

Request JSON

```
{
  "is_active": true
}
```

Frontend notes: Edits apply only to future generation and never rewrite historical tasks or results. Do not offer unsupported custom cron, weekday, or biweekly controls.

POST **/api/v2/operations/schedules/{schedule_id}/deactivate/**

Purpose: Deactivate future execution while preserving schedule/task history.

Response JSON

```
{
  "success": true,
  "data": {
    "id": 17,
```

```
    "is_active": false
  }
}
```

Frontend notes: Use deactivation rather than hard delete. GET /api/v2/operations/schedules/ and GET /api/v2/operations/schedules/{schedule_id}/ are farm-scoped reads.

next_run_at is backend-authoritative and recurrence is calculated in Africa/Lagos time. Display the returned offset-aware timestamp; do not calculate recurrence in the frontend. Automatic generation is server controlled and occurrence-idempotent. A monthly recurrence uses the last valid day where a calendar date does not exist (for example, January 31 advances to February 28/29). On reactivation, the backend creates one missed occurrence then advances to the next future occurrence; it does not backfill every missed interval. If a stored assignee is no longer eligible, the backend deactivates the schedule without generating unauthorized work.

12. My Work and field execution (D02-008)

My Work is the signed-in user's server-scoped personal execution inbox. Use GET /api/v2/operations/my-work/ for paginated open tasks assigned to that user, and GET /api/v2/dashboard/my-work/ for the backend-authoritative Open, Today, Overdue, Completed Today, and Upcoming summary. Today and Overdue do not overlap; unable-to-complete work is tracked in the Operations exceptions queue. Never construct My Work by downloading a farm-wide task list and filtering it in the client.

GET	/api/v2/operations/my-work/?farm_id=1

Purpose: Fetch only the current user's open assigned tasks in an authorized farm.

Response JSON

```
{
  "success": true,
  "data": [
    {
      "id": 304,
      "title": "Vaccinate COW-001",
      "subject": {
        "type": "animal",
        "id": 42,
        "label": "COW-001"
      },
      "status": "assigned",
      "is_overdue": false
    }
  ],
  "meta": {
    "pagination": {
      "page": 1
    }
  }
}
```

Frontend notes: farm_id narrows access only. Subject, priority, due_at and lifecycle status are distinct data dimensions.

Use GET /api/v2/users/me/tasks/ for the personal All Tasks/history view, with optional status and farm_id filters. Use GET /api/v2/users/me/activity/ for the authenticated user's activity timeline, optionally narrowed by farm_id. Both are server-scoped and must not be replaced with client-side aggregation.

Task actions remain contextual: Accept acknowledges work; Start is optional; Complete uses the task-type-specific domain form and authoritative backend transaction; Unable to Complete sends a canonical reason code. Cancel requires cancel_operation and is not granted merely by assignment. Supported retries must reuse the same client_request_id. Completed tasks leave the open inbox.

My Work is user-centric rather than role-name-centric, but channel entitlement remains enforced. It is personal work, not a manager operations workspace or Quick Record path. Offline mobile records remain pending until backend revalidation succeeds; show sync/conflict state rather than claiming final completion locally.

13. Mobile JWT API

The mobile client uses JWT Bearer authentication, not browser cookies. Mobile-only Swagger/OpenAPI is available at /mobile/api/docs and /mobile/api/openapi.json. Send Authorization: Bearer <access-token>; the token must belong to an active mobile-channel session.

Mobile endpoints are limited to field execution: GET /mobile/api/work/, GET /mobile/api/work/dashboard/, GET /mobile/api/tasks/{task_id}/, and POST task accept, start, complete, and unable-to-complete actions. They revalidate current mobile channel, account state, farm access, task relationship, capability, lifecycle, and typed domain state on every request.

For offline retries, retain the same client_request_id for completion and unable-to-complete submissions. A local queued record is pending until the backend accepts it; do not present it as authoritative completion before synchronization.

14. Manager Operations Dashboard (D02-009)

Use GET /api/v2/operations/dashboard/ for the authorized operational control summary. It is not My Work and does not return a second task model. Optional farm_id may narrow scope only; an unauthorized farm returns the standard farm-access error.

GET	/api/v2/operations/dashboard/?farm_id=1

Purpose: Fetch server-authoritative operational progress metrics.

Response JSON

```
{
  "success": true,
  "data": {
    "completed": 60,
    "actionable": 35,
    "due_today": 8,
    "overdue": 5,
    "completion_percentage": 63.16,
    "team_progress": {
      "completed": 60,
      "total": 95,
      "percentage": 63.16
    }
  },
  "meta": {
    "evaluation_time": "2026-09-23T14:30:00+01:00",
    "timezone": "Africa/Lagos",
    "farm_scope": 1
  }
}
```

Frontend notes: Do not calculate these metrics in the frontend. Due Today is actionable work from evaluation_time until the next Lagos day boundary; overdue is actionable work before evaluation_time, so the populations do not overlap.

For users without management capability, the server retains personal assigned-work scope. The endpoint is read-only and creates no tasks, events, schedules, or domain records.

The management Task list at GET /api/v2/operations/tasks/ supports server-side search, farm, status, work type, assignee, due_from/due_to dates, priority, and deterministic sorting (due_asc, due_desc, priority, recent). My Work remains separately scoped to the signed-in assignee. The My Work dashboard summary returns Open, Today, Overdue, Completed Today, and Upcoming counts; Today excludes already overdue tasks, and unable-to-complete tasks are shown through the Operations exceptions queue rather than counted as open work.

15. Farm and historical-data isolation (D02-010)

D02-010 is product locked. A direct object outside the current authorized Organization/Farm/data scope is concealed as 404 with the relevant resource-specific *_NOT_FOUND code (for example, FARM_NOT_FOUND); do not reveal its existence. Return 403 PERMISSION_DENIED only when the object is within the user's visible Farm scope but the requested action is not permitted.

On the next successful mobile synchronization, call GET /mobile/api/sync-scope/ and remove operational cache data belonging only to Farm IDs absent from authorized_farm_ids. Cached data is never authoritative; queued writes are reauthorized by the server and can be rejected after access revocation.

Farm deactivation preserves identity and operational history. Do not offer hard deletion for a Farm with historical Tasks, results, events, schedules, or assignments. After an Animal moves Farm, historical records retain their original Farm scope; a Farm B-only user cannot access Farm A history, while dual-Farm and Organization-wide users may access their authorized history.

GET	/mobile/api/sync-scope/

Purpose: Fetch the authoritative current Farm scope before reconciling offline operational cache.

Response JSON

```
{
  "success": true,
  "data": {
    "authorized_farm_ids": [
      1,
      3
    ],
    "cache_policy": "remove_revoked_farm_operational_data"
  }
}
```

Frontend notes: Remove cached operational records exclusively belonging to Farm IDs not returned. This endpoint may be called with a live mobile session after Farm access has been revoked so the client can safely reconcile local state.

16. Events, audit and activity (D02-011)

Material Operations actions create immutable, structured canonical events. The frontend must treat the returned current Task state as current state only; event history is the authority for how that state was reached.

Web clients continue using the secure-cookie flow. Mobile clients continue using JWT Bearer tokens only under /mobile/api/.

17. Notifications and alerts (D02-012)

Use GET /api/v2/notifications/ for the authenticated recipient's Notification Center and GET /api/v2/notifications/unread-count/ for its matching badge. Mark only the signed-in recipient's records read through POST /api/v2/notifications/{notification_id}/read/ or /read-all/. Use notification_type and the authoritative reference, not title/body text; opening the reference rechecks current authorization.

Alerts are condition projections, not Task statuses or notification substitutes. Existing farm/health alert and Operations exception resources remain the authoritative attention views. No due-soon threshold, escalation, severity, external delivery, SMS, WhatsApp, or blanket completion notification policy is implied.

18. Error handling and retry behavior

```
{
  "success": false,
  "code": "FARM_NOT_FOUND",
  "message": "Farm could not be found.",
  "data": null,
  "errors": {},
  "retryable": false
}
```

Handle codes, not message text. Recommended frontend actions:

AUTHENTICATION_REQUIRED / SESSION_EXPIRED → refresh once or redirect to login. ACCOUNT_DEACTIVATED → sign out and show account notice. FARM_NOT_FOUND for an inaccessible scope → show a neutral unavailable state without implying that the object exists. PERMISSION_DENIED / CHANNEL_ACCESS_DENIED → show access-denied state. VALIDATION_ERROR → show field errors. CONFLICT → preserve the server result or ask the user to resolve the conflict.

19. Implementation evidence

Implementation evidence is maintained under api_dev/reports/evidence/.

20. Implementation checklist

✓ Send X-App-Channel. ✓ Use secure cookies. ✓ Use capability booleans. ✓ Read the standard envelope. ✓ Never allow frontend-only tenant filtering.

Contract reference: FarmOS Frontend API Contract v2.2, including implemented D02-007 through D02-010 and accepted foundations for D02-011 through D02-014. Backend base URL for Docker development: http://127.0.0.1:8082.

21. Open policy decisions and implementation blockers

The following behaviour is intentionally not inferred by the development backend or Frontend. It requires management approval before implementation or integration. D02-010 is implemented and product locked; it is not an open-policy item.

D02-011: overdue-event generation, offline timeline ordering, event retention, and final dedicated activity route.

D02-012: due-soon threshold, overdue escalation, severity model, persisted-alert decision, preferences, retention, external channels, and final notification-route alignment.

D02-013: idempotency retention, record version/ETag contract, offline conflict treatment, bulk/sync-batch policy, and engineering alert thresholds.

D02-014: completion/on-time/overdue-rate definitions, reporting timezone, duration rules, export formats/jobs, and historical-label policy.

Until resolved, clients must rely on the currently documented endpoints and must not calculate or emulate these policies locally.

22. Development correction addendum (2026-10-06)

These additions are available on the development backend at http://127.0.0.1:8001. They retain the v2 envelope and farm authorization rules. Render actions from effective capabilities, handle concealed cross-farm 404s, and do not treat these development changes as live or frontend-accepted.

Animal groups and membership history

Groups are farm-scoped. Membership removal is soft/history-preserving; rejoining creates a new membership row. Cross-farm animal membership is rejected. Use active membership for current group contents and include_removed=true only for history.

GET `/api/v2/groups/?farm_id=1`

Purpose: List active groups in an authorized farm; use include_inactive=true to include inactive groups.

POST `/api/v2/groups/`

Purpose: Create a group.

Request JSON

```
{
  "farm_id": 1,
  "name": "Milking",
  "description": "Morning milking group"
}
```

Response JSON

```
{
  "success": true,
  "data": {
    "id": 12,
    "farm_id": 1,
    "name": "Milking",
    "status": "ACTIVE",
    "member_count": 0
  }
}
```

Frontend notes: Requires animal-create capability.

GET `/api/v2/groups/{group_id}/members/?include_removed=false`

Purpose: Read current members or membership history.

Response JSON

```
{
  "success": true,
  "data": [
    {
      "membership_id": 21,
      "animal_id": 42,
      "tag_id": "COW-042",
      "status": "ACTIVE",
      "joined_at": "2026-10-06T10:00:00+01:00",
      "removed_at": null
    }
  ]
}
```

POST `/api/v2/groups/{group_id}/members/{animal_id}/`

Purpose: Add an animal to an active same-farm group.

Response JSON

```
{
  "success": true,
  "data": {
    "id": 21,
    "animal_id": 42,
    "group_id": 12
```

```
  }
}
```

DELETE `/api/v2/groups/{group_id}/members/{animal_id}/`

Purpose: Remove current membership without deleting its history.

Farm labels and animal gallery

Labels are optional classifications, not authoritative lifecycle or health states. Reserved state names such as Sick, Pregnant, Quarantine, Dead, and Sold are rejected. Label listing supports search and returns assigned animal_count; assignment is farm-bound. Group listing supports status=active|inactive|all, search, active member counts, and updated_at. Group member listing supports server-side search and species, breed, gender, and lifecycle filters and includes the standard animal card for selector/list reuse. The profile_incomplete animal filter uses the same six-field rubric returned in each animal card: identification tag, species, breed, date/estimated age, housing, and source. Duplicate group or label renames return CONFLICT. Gallery uploads retain multiple images and expose a primary image; use multipart form data for uploads.

GET `/api/v2/settings/labels/?farm_id=1`

Purpose: List active farm labels; include_inactive=true returns inactive labels.

POST `/api/v2/settings/labels/`

Purpose: Create a farm label.

Request JSON

```
{
  "farm_id": 1,
  "name": "Priority",
  "color": "#D97706",
  "description": "Needs follow-up"
}
```

POST `/api/v2/settings/animals/{animal_id}/labels/{label_id}/`

Purpose: Assign a same-farm label to an animal.

GET `/api/v2/animals/{animal_id}/media/`

Purpose: List an animal's gallery media, including primary and capture metadata.

POST `/api/v2/animals/{animal_id}/media/`

Purpose: Upload an image using multipart/form-data fields file, note, make_primary, and optional captured_at.

Frontend notes: If make_primary is true, the backend updates the legacy profile image pointer too. Keep the returned media URL; do not assume a local file path.

POST `/api/v2/animals/{animal_id}/media/{media_id}/primary/`

Purpose: Set an existing gallery item as the primary image.

Health observations and cases

An observation is not a diagnosis. A healthy active animal with a new observation becomes at_risk; opening/escalating a clinical case may set it to sick. Clinical assessment requires manage_health_case. Use the returned status and case_id rather than deriving status in the UI.

POST `/api/v2/health/observations/`

Purpose: Record an observation.

Request JSON

```
{
  "farm_id": 1,
  "animal_id": 42,
  "symptoms": "Reduced appetite",
  "severity": "mild",
  "create_case": false,
  "create_task": false
}
```

Response JSON

```
{
  "success": true,
  "data": {
    "id": 88,
    "status": "open",
    "animal_id": 42,
    "case_id": null
  }
}
```

POST **/api/v2/health/observations/{observation_id}/acknowledge/**

Purpose: Acknowledge an open observation; the response status becomes acknowledged.

POST **/api/v2/health/observations/{observation_id}/assess/**

Purpose: Resolve or escalate an open/acknowledged observation.

Request JSON

```
{
  "outcome": "escalate_to_health_case",
  "note": "Clinical review required",
  "case_title": "Possible infection"
}
```

Response JSON

```
{
  "success": true,
  "data": {
    "status": "escalated_to_health_case",
    "case_id": 19
  }
}
```

POST **/mobile/api/health/cases/**

Purpose: JWT-mobile case creation using the same farm-scoped case service.

Request JSON

```
{
  "farm_id": 1,
  "animal_id": 42,
  "title": "Possible infection",
  "notes": "Field report"
}
```

Schedules and mobile field API

Schedule create/patch now accepts starts_at, ends_at, and priority. Schedule responses include Farm, target, default assignee, and creator context alongside their IDs. next_run_at must fall within the configured window; a finished window stops further generation. Existing generated tasks are not rewritten. Animal list filters are shared by web and mobile, including labels, profile completeness, open observations/cases, age ranges, and supported sort keys. Animal search also matches saved profile names; sort=name_asc or sort=name_desc orders by saved name with deterministic tag/id fallbacks. Unsupported sort values return a structured 422 validation error.

Task target selectors return paginated, current-farm eligible Animals with tag, saved display name (falling back to tag), image, species, breed, gender, health/lifecycle status, housing, and current Group context. Animal target search matches tag, notes, and saved name. Group targets include current active member count. The web and mobile eligible-assignee selectors accept search=, page=, and page_size= (default 20) and return only active current-farm users who satisfy the work-type capability checks; results include display name, email, avatar URL, and Farm context. Search is applied

server-side before capability checks, and pagination keeps selector responses bounded.

GET `/api/v2/operations/schedules/{schedule_id}/tasks/?page=1&page;_size=20`

Purpose: List paginated generated Task history for an authorized Schedule; each Task retains the Schedule source reference.

POST `/api/v2/operations/schedules/`

Purpose: Create a scheduled work template with an optional window and priority.

Request JSON

```
{
  "farm_id": 1,
  "task_type": "vaccination",
  "title": "Seasonal vaccination",
  "recurrence": "monthly",
  "starts_at": "2026-10-06T09:00:00+01:00",
  "ends_at": "2027-03-31T17:00:00+01:00",
  "next_run_at": "2026-10-10T09:00:00+01:00",
  "priority": "high"
}
```

Response JSON

```
{
  "success": true,
  "data": {
    "starts_at": "2026-10-06T09:00:00+01:00",
    "ends_at": "2027-03-31T17:00:00+01:00",
    "priority": "high"
  }
}
```

GET `/mobile/api/animals/?farm_id=1&open;_observation=true`

Purpose: List authorized animals using the shared server-side filter and sort contract.

GET `/mobile/api/work-types/{work_type}/eligible-targets/?farm_id=1■_type=animal`

Purpose: Fetch eligible targets for a work type; only active and authorized subjects are returned.

GET `/mobile/api/work-types/{work_type}/eligible-assignees/?farm_id=1&search;=Ada&page;=1&page;_size=20`

Purpose: Search and page through mobile-enabled, farm-scoped assignees with required capabilities.

POST `/mobile/api/quick-record/`

Purpose: Create a direct domain record without a placeholder task. Send a client_request_id for retry-safe submission.

Request JSON

```
{
  "farm_id": 1,
  "task_type": "observation",
  "animal_id": 42,
  "symptoms": "Reduced appetite",
  "client_request_id": "field-observation-001"
}
```

GET `/mobile/api/notifications/`

Purpose: Read the authenticated mobile user's notification inbox; unread count and mark-read routes are also available.

Frontend acceptance checklist: verify animal gallery list/upload/primary selection (including upload errors and refresh); mobile quick-record success, duplicate retry using the same client_request_id, and denied farm/capability access; schedule create/edit around starts_at and ends_at, out-of-window validation, generated-task history pagination, and inactive/ended schedules; group vaccination roster paging, ineligible rows, assignment restrictions, and completion-time

revalidation. Record frontend build, environment, device/browser, scenario result, and defects. These scenarios are not claimed as passed by backend tests.

Backend verification includes 48/48 main HTTP E2E checks, 28/28 v2.2 verifier checks, and 74/74 common contract tests against an isolated development test database (2026-10-13). Focused regressions also passed for schedule history/context, work selectors, animal name search/sort, and web/mobile group vaccination roster. Django system checks and migration-drift checks pass. See api_dev/reports/evidence/backend-correction-frontend-handoff-2026-10-06.md for the requirement-to-test matrix and run details. This is backend evidence only; frontend rendering, user acceptance, and promotion remain separate.